



GERENCIAMENTO DE TRANSAÇÕES COM JPA E HIBERNATE

As implementações realizadas durante as aulas utilizam `EntityManager`, `EntityManagerFactory` e uma transação local controlada explicitamente por `getTransaction().begin()` e `commit()`.

A partir desse exemplo, é possível avançar de uma visão meramente operacional — “iniciar e finalizar uma transação” — para uma compreensão mais importante para a formação profissional: **uma transação representa uma unidade de trabalho que precisa possuir início, processamento, tratamento de falhas e encerramento bem definidos.**

Essa perspectiva é coerente tanto com a especificação Jakarta Persistence quanto com práticas consolidadas no ecossistema Hibernate e com o padrão **Unit of Work**, de Martin Fowler.

O que realmente significa controlar uma transação?

É comum que, ao aprender JPA, se associe uma transação simplesmente à sequência:

```
<<> Java  
  
em.getTransaction().begin();  
  
// alterações  
  
em.getTransaction().commit();
```

Embora tecnicamente correta, essa visão é incompleta. Uma transação deve ser entendida como a **fronteira de uma operação que precisa ser tratada como uma unidade lógica e atômica.**

Por exemplo, suponha uma operação de transferência bancária:

Debitar conta A → Creditar conta B → Registrar operação

Não seria aceitável concluir apenas metade da operação. Se o débito ocorrer e o crédito falhar, o sistema precisa retornar ao estado anterior:

INÍCIO → Débito → Crédito → Commit → FIM

Ou, diante de uma falha

INÍCIO → Débito → Erro → Roolback → FIM

É justamente essa ideia que se aproxima do padrão **Unit of Work**: reunir as alterações relacionadas a uma operação de negócio e coordenar sua persistência como uma unidade.

Commit e Roolback

O `commit()` e o `rollback()` representam o encerramento bem-sucedido da unidade transacional. Enquanto um efetiva a modificação que está sendo realizada no banco, o outro despreza aquilo que se pretendia realizar.

De modo geral, a aplicação trabalha com objetos gerenciados; o JPA/Hibernate coordena a sincronização desses objetos com o banco dentro da unidade transacional.

Abaixo apresento uma nova versão do código que implementamos anteriormente para realizar a atualização de registros no banco aplicando as boas práticas relativas ao controle de transações.

```

public class UpdateCT {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("jpa");
        EntityManager em = emf.createEntityManager();

        String novoNome = "Produto Novo";
        Integer novaCategoria = 10;

        EntityTransaction tx = em.getTransaction();

        try {

            // 1. Início
            tx.begin();

            // 2. Unidade de trabalho
            Produto produto = em.find(Produto.class, 3);

            if (produto == null) {
                throw new RuntimeException("Produto não encontrado.");
            }

            produto.setNome(novoNome);
            produto.setCategoria(novaCategoria);

            // 3. Finalização
            tx.commit();

        } catch (RuntimeException e) {

            // 4. Falha
            if (tx.isActive()) {
                tx.rollback();
            }

            throw e;

        } finally {

            // 5. Liberação dos recursos
            em.close();

        }

        emf.close();
    }
}

```

Esse padrão é particularmente importante porque evita deixar o controle da transação dependente exclusivamente do caminho de sucesso. A documentação oficial da Jakarta Persistence apresenta justamente a combinação `try/catch/finally`, com `rollback()` caso a transação esteja ativa e `close()` no encerramento do `EntityManager`.

O exemplo demonstra ainda como evitar a ocorrências transações presas ou não finalizadas.

Uma transação aberta por tempo excessivo pode causar problemas muito maiores do que uma simples exceção Java. Ela pode manter: locks; conexões ocupadas; recursos do banco; objetos gerenciados; operações pendentes; concorrência reduzida; filas para outras transações.

Concluindo

Gerenciamento de transações não consiste simplesmente em chamar `begin()` e `commit()`. Consiste em definir, controlar e encerrar corretamente uma unidade de trabalho que preserva a consistência da aplicação e do banco de dados.